
Exercício sobre Decorador

Padrões de Projeto

Engenharia de Software II

A herança é uma forma comum de estender comportamentos. O conceito é simples - criar uma classe base e estendê-la conforme necessário em suas subclasses. Mas isso torna as extensões fortemente ligadas à classe base (alto acoplamento) e pode gerar uma explosão de subclasses.

O Padrão Decorador representa uma alternativa à herança quando se precisa adicionar novas funcionalidades em uma classe base. Como o nome sugere, o padrão decorador está associado à decoração de algo - objetos, para ser exato.

Em vez de usar herança, usa-se composição (mais precisamente, agregação em UML) para adicionar tais funcionalidades dinamicamente nas classes base. Portanto, Decorador é um exemplo de aplicação de um princípio de projeto de padrões de projeto: **Prefira Composição a Herança**.

Implemente a solução para o problema apresentado a seguir em linguagem de programação Java usando o Padrão Decorador. Envie o código e o diagrama de classes da solução.

PROBLEMA:

Suponha que você foi a uma loja de carros e encomendou um carro com um opcional, que é o sistema de navegação. Durante o pedido, você quis trocar a cor “padrão” do carro para a cor azul. O atendente informou que isso era possível, porém seria necessário descartar o pedido anterior e iniciar um novo, agora para um carro com sistema de navegação e cor azul.

Além disso, o atendente explicou que cada opcional impacta diretamente no preço final do veículo. Por exemplo, o sistema de navegação adiciona um custo adicional, assim como a escolha de uma cor especial também altera o valor do carro. Dessa forma, cada combinação de opcionais resulta em um preço diferente.

Ao finalizar o seu pedido, um outro cliente chegou e solicitou um carro da cor azul, porém sem o sistema de navegação. Para sua surpresa, o funcionário informou que não seria possível atender a esse pedido, pois no sistema só existia a opção de carro azul com navegação.

Esse comportamento evidencia uma limitação no sistema de pedidos, que utiliza a estrutura de classes apresentada na Figura 1.

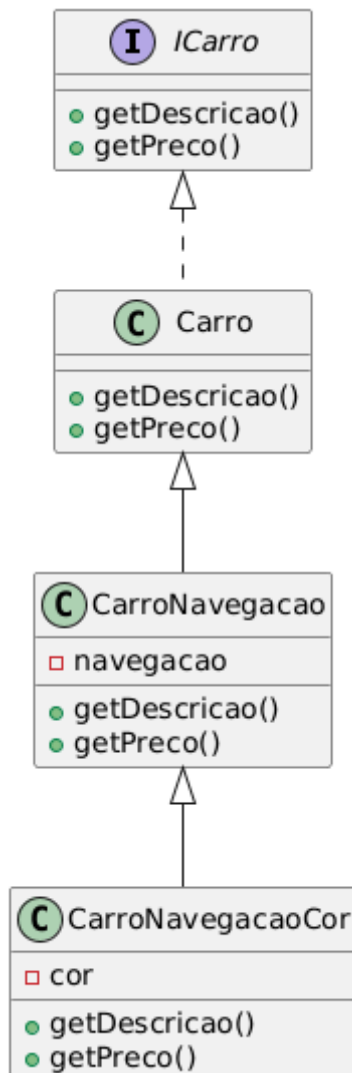


Figura 1 - Diagrama de classes original, sem utilizar padrões de projeto.

Observe que há uma interface ICarro, que define os comportamentos de um carro. Inicialmente, considera-se que um carro possui métodos como getDescricao() e getPreco(), responsáveis por descrever o veículo e retornar seu valor.

Existem classes como Carro e CarroNavegacao, sendo que esta última estende a primeira ao adicionar o atributo de navegação, impactando tanto a descrição quanto o preço do veículo. Além disso, há a classe CarroNavegacaoCor, que estende CarroNavegacao ao incluir o atributo cor, também influenciando o valor final.

Devido a essa estrutura baseada em herança, não foi possível atender ao segundo cliente, pois não existe uma classe que represente um carro com cor, mas sem navegação.

Além disso, essa abordagem não escala bem. Cada novo opcional (por exemplo, “Teto Solar”, “Bancos de Couro”, “Sensor de Estacionamento”) exige a criação de novas subclasses para representar todas as combinações possíveis. Isso gera uma explosão de classes, além de duplicação de lógica, especialmente no cálculo do preço, que precisa ser reimplementado ou ajustado em várias subclasses.

Portanto, considerando que os opcionais são independentes entre si, essa abordagem apresenta dois problemas principais:

O cliente precisa definir todas as características do carro no momento da instanciação. Não é possível adicionar ou remover opcionais dinamicamente, nem recalculer o preço de forma incremental.

A adição de novos opcionais exige modificações extensivas no sistema, com a criação de múltiplas subclasses para cobrir todas as combinações, tornando o código difícil de manter, evoluir e validar, especialmente no que diz respeito ao cálculo correto do preço.

O código poderia ser escrito em Java conforme apresentado a seguir.

```
Java
interface ICarro{
    String getDescricao();
    double getPreco();
}

class Carro implements ICarro {
    Carro() {
        System.out.println("Criando um carro");
    }

    public String getDescricao() {
        return "Carro";
    }

    public double getPreco() {
        return 50000.0;
    }
}

class CarroNavegacao extends Carro {
    private String navegacao;

    CarroNavegacao() {
        this.navegacao = "Navegação";
    }
}
```

```

    public String getDescricao() {
        return super.getDescricao() + " " + navegacao;
    }

    public double getPreco() {
        return super.getPreco() + 2500.0;
    }
}

class CarroNavegacaoCor extends CarroNavegacao {
    private String cor;

    CarroNavegacaoCor(String cor) {
        this.cor = cor;
    }

    public String getDescricao() {
        return super.getDescricao() + " " + cor;
    }

    public double getPreco() {
        return super.getPreco() + 1800.0;
    }
}

public class MainProblema {
    public static void main(String[] args) {

        // Cliente 1: carro com navegação
        ICarro carro = new CarroNavegacao();
        System.out.println("Pedido inicial:");
        System.out.println(carro.GetDescricao());
        System.out.println("Preço: R$ " + carro.GetPreco());

        System.out.println("\nCliente decide adicionar cor azul...");
        System.out.println("-> Precisa descartar o objeto e criar outro!\n");

        // Recriação do objeto : problema
        carro = new CarroNavegacaoCor("Azul");

        System.out.println("Novo pedido:");
        System.out.println(carro.GetDescricao());
        System.out.println("Preço: R$ " + carro.GetPreco());

        System.out.println("\n-----\n");

        // Cliente 2: quer só cor (sem navegação)
        System.out.println("Outro cliente quer APENAS cor azul (sem navegação)...");

        System.out.println("-> Não existe classe para isso no sistema!");
        System.out.println("-> Não é possível atender o pedido.\n");
    }
}

```

SOLUÇÃO: O padrão Decorator

Em cenários nos quais um objeto precisa ser estendido dinamicamente, e cada extensão representa uma funcionalidade independente (como opcionais de um carro), torna-se necessário adotar uma abordagem mais flexível do que a herança tradicional.

Uma alternativa é estruturar o sistema de forma que novas responsabilidades possam ser adicionadas aos objetos **em tempo de execução**, sem a necessidade de criar subclasses para cada combinação possível de funcionalidades.

O Padrão Decorator atende a esse objetivo ao permitir que objetos sejam **envolvidos por outros objetos** que adicionam novos comportamentos. Dessa forma, cada funcionalidade adicional (por exemplo, sistema de navegação, cor especial ou outros opcionais) pode ser tratada como uma extensão independente, aplicada conforme a necessidade.

Em termos formais, o padrão Decorator permite adicionar responsabilidades adicionais a um objeto dinamicamente, sem modificar sua implementação original e sem impactar outras funcionalidades já existentes.

Nessa abordagem, as extensões são organizadas de modo que possam ser combinadas entre si, possibilitando a construção de diferentes configurações de objetos de forma incremental. Isso é especialmente relevante em cenários onde cada funcionalidade também influencia outros aspectos do sistema, como o **cálculo do preço final**, que passa a ser composto gradualmente à medida que novas responsabilidades são adicionadas.

Do ponto de vista estrutural, essa solução envolve o uso de um tipo comum que representa o objeto principal e suas extensões. As extensões mantêm uma referência para esse tipo, permitindo que o comportamento original seja preservado e ampliado progressivamente.

Com isso, torna-se possível criar objetos com diferentes combinações de funcionalidades sem a necessidade de definir previamente todas as variações em tempo de projeto, reduzindo o acoplamento e melhorando a extensibilidade do sistema.

O diagrama de classes UML para a implementação do padrão Decorator é apresentado na Figura 2.

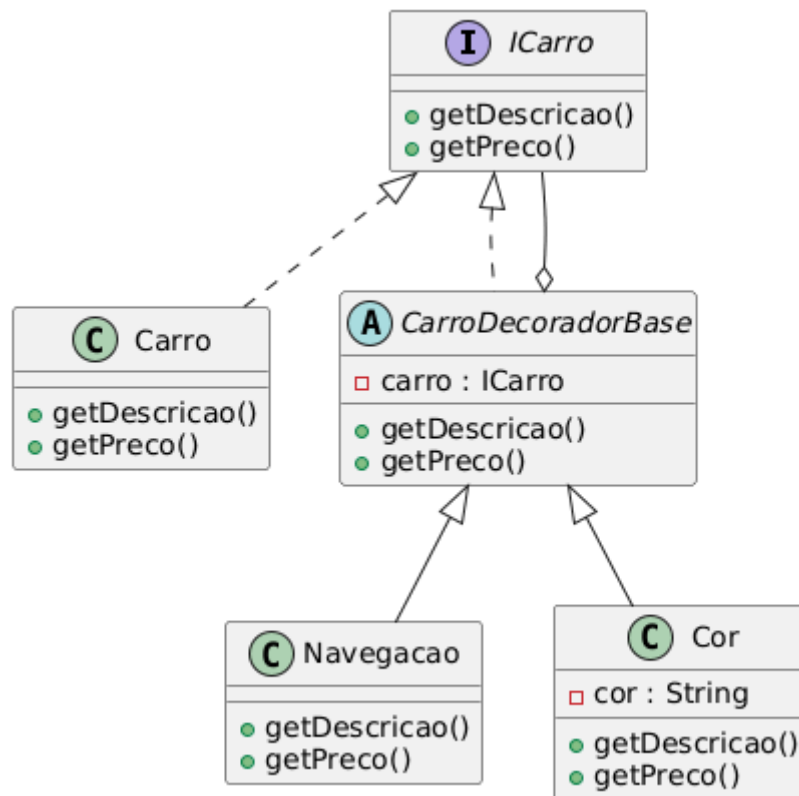


Figura 2 - Diagrama de classes UML para implementar o padrão Decorador.